VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS FAKULTETAS

INFORMATIKOS STUDIJŲ PROGRAMA

Kursinio darbo projektas

Euristinių algoritmų ir globaliųjų optimumų lyginamoji analizė konkrečiam VRPPD uždaviniui.

Comparative analysis of heuristic algorithms and global optima for the specific VRPPD problem.

Aleksandras Šukelovič

Darbo vadovas : Adomas Birštunas

Vilnius
2026

Santrauka

Šiame kursinio darbo projekte nagrinėjama transporto priemonių maršrutizavimo problema su paėmimais ir pristatymais (VRPPD). Pagrindinis darbo tikslas – realizuoti ir palyginti skirtingus algoritmus, skirtus optimizuoti logistikos maršrutus pagal kelis kriterijus: tuščią ridą, bendrą atstumą ir transportavimo kainą.

Darbe sukurta programinė įranga, apimanti duomenų generavimo modulį bei du sprendimo metodus: tikslųjį pilno perrinkimo (angl. *Brute Force*) algoritmą ir euristinį lygiagretųjį imituoto atkaitinimo (angl. *Parallel Simulated Annealing*, p-SA) algoritmą. Pilno perrinkimo metodas, realizuotas pasitelkiant *Rust* kalbą, naudojamas kaip etalonas (angl. *Ground Truth*) algoritmų tikslumui vertinti mažos apimties uždaviniuose. Euristinis p-SA algoritmas, naudojantis RCRS (angl. *Residual Capacity and Radial Surcharge*) metodą pradiniam sprendiniui generuoti, pritaikytas didesnės apimties uždaviniams spręsti lygiagrečioje aplinkoje.

Atliktas eksperimentinis tyrimas parodė, kad euristinis algoritmas geba rasti sprendinius su vidutine santykine paklaida, neviršijančia 1% vidutinio dydžio uždaviniams. Taip pat atlikta daugiakriterinė analizė atskleidė esminius kompromisus (Pareto aibės aproksimaciją) tarp kainos, atstumo ir tuščios ridos, parodant, kad trumpiausias maršrutas nebūtinai yra ekonomiškai efektyviausias.

Raktiniai žodžiai: VRPPD, transporto optimizavimas, pilnas perrinkimas, imituotas atkaitinimas, p-SA, RCRS, daugiakriterinis optimizavimas.

Abstract

This coursework project addresses the Vehicle Routing Problem with Pickups and Deliveries (VRPPD). The main objective is to implement and compare different algorithms designed to optimize logistics routes based on multiple criteria: empty mileage, total distance, and transportation cost.

The software developed includes a data generation module and two solution methods: an exact Brute Force algorithm and a heuristic Parallel Simulated Annealing (p-SA) algorithm. The Brute Force method, implemented in Rust, serves as a baseline (Ground Truth) for evaluating algorithm accuracy on small-scale instances. The heuristic p-SA algorithm, utilizing the RCRS (Residual Capacity and Radial Surcharge) method for initial solution generation, is adapted for solving larger-scale problems in a parallel computing environment.

Experimental research demonstrated that the heuristic algorithm finds solutions with an average relative percentage deviation not exceeding 1% for medium-sized problems. Furthermore, a multi-objective analysis revealed essential trade-offs (Pareto set approximation) between price, distance, and empty mileage, indicating that the shortest route is not necessarily the most cost-effective one.

Keywords: VRPPD, transport optimization, Brute Force, Simulated Annealing, p-SA, RCRS, multi-objective optimization.

Turinys

Santrauka **2**

Abstract **3**

Santrumpos ir terminai **6**

Įvadas **7**

1. Problemos aprašymas **8**

2. Duomenų generavimas **9**
2.1. Pirminių duomenų paruošimas 9
2.2. Uždavinių generavimas 9

3. Pilno perrinkimo algoritmas **10**
3.1. Algoritmo logika ir matematinis pagrindas 10
3.2. Realizacija ir optimizacija (NodeJS ir Rust) 10
3.2.1. NodeJS realizacija 11
3.2.2. Rust realizacija ir validacija 11

4. Lygiagretusis imituoto atkaitinimo algoritmas **12**
4.1. Pradinio sprendinio generavimas (RCRS) 12
4.2. Lygiagretinimo strategija 12
4.3. Kaimynystės operatoriai 13
4.3.1. Perkėlimo operatorius (Shift) 13
4.3.2. Sukeitimo operatorius (Swap) 13
4.3.3. Vidinis sumaišymas (Intra-Shuffle) 14
4.4. Parametrų derinimas (Tuning) 14

5. Algoritmo pritaikymas ir pseudokodas **16**
5.1. Kaimynystės operatorių adaptacija 16
5.2. Lygiagrečios sąveikos logika (Re-heating) 16
5.3. Algoritmo pseudokodas 16

6. Eksperimentiniai rezultatai **18**
6.1. Tyrimo metodika ir imtis 18
6.2. Algoritmų našumas ir laiko sąnaudos 18
6.3. Sprendinių kokybės analizė 19
6.4. Algoritmo patikimumas 20
6.5. Konvergavimo charakteristikos 20
6.6. Daugiakriterinė analizė 21

7. Tolimesni darbai **24**

8. Išvados ir rezultatai **25**

Santrumpos ir terminai

Darbe naudojamos santrumpos ir terminai:

VRPPD (angl. *Vehicle Routing Problem with Pickups and Deliveries*). Transporto priemonių maršrutizavimo problema, kurioje prekės turi būti paimtos iš tam tikrų vietų ir pristatytos į kitas, laikantis talpos ir eiliškumo apribojimų.

p-SA (angl. *Parallel Simulated Annealing*). Lygiagretusis imituoto atkaitinimo algoritmas. Metaeuristinis optimizavimo metodas, imituojantis metalų aušinimo procesą, vykdomas keliose gijose vienu metu, siekiant išvengti lokalių optimumų.

RCRS (angl. *Residual Capacity and Radial Surcharge*). Likutinės talpos ir radialinio antkainio euristika. Konstruktyvus metodas pradiniam maršrutui sudaryti, atsižvelgiant į atstumą ir likusią transporto priemonės talpą.

Brute Force (liet. *Pilnas perrinkimas*). Algoritmas, kuris sistemingai patikrina visas įmanomas kandidatų kombinacijas, kad rastų garantuotai geriausią sprendinį. Dėl eksponentinio sudėtingumo taikomas tik mažiems uždaviniams.

RPD (angl. *Relative Percentage Deviation*). Santykinis procentinis nuokrypis. Metrika, naudojama įvertinti, kiek euristinio algoritmo rastas sprendinys skiriasi nuo žinomo optimalaus sprendinio.

NP-sunkus (angl. *NP-hard*). Uždavinių sudėtingumo klasė. Tai uždaviniai, kuriems (šiuo metu žiniomis) neegzistuoja efektyvūs algoritmai, randantys tikslų atsakymą per polinominį laiką.

Tuščia rida (angl. *Empty Mileage*). Atstumas, kurį transporto priemonė nuvažiuoja be krovinio (pvz., vykdama į pirmąjį paėmimo tašką arba važiuodama tarp pristatymo ir kito paėmimo taškų).

Pareto aibė. Daugiakriteriniame optimizavime – aibė sprendinių, kurių negalima pagerinti pagal vieną kriterijų, nepabloginant bent vieno kito kriterijaus.

Įvadas

Transporto ir logistikos sektoriuje maršrutų optimizavimas yra pagrindinis veiksnys, lemiantis veiklos efektyvumą, kaštų mažinimą ir konkurencingumą. Klasikinė transporto priemonių maršrutizavimo problema (angl. *Vehicle Routing Problem*, VRP) ir jos variacijos, tokios kaip VRP su paėmimais ir pristatymais (angl. *VRP with Pickups and Deliveries*, VRPPD), priklauso NP-sunkių (angl. *NP-hard*) uždavinių klasei [GJ90]. Didėjant užsakymų skaičiui, galimų sprendinių erdvė plečiasi eksponentiškai, todėl rasti globalų optimumą per priimtiną laiką naudojant tiksliuosius metodus tampa neįmanoma.

Šio kursinio darbo projekto tikslas – sukurti ir ištirti programinę įrangą, skirtą VRPPD uždavinių sprendimui, lyginant tiksliuosius ir euristinius algoritmus bei analizuojant skirtingų optimizavimo kriterijų įtaką galutiniam rezultatui.

Siekiant šio tikslo, sprendžiami šie uždaviniai:

1. **Duomenų generavimas:** Sukurti testavimo duomenų rinkinį, paremtą realiais logistikos srautais ir sugeneruoti įvairaus sudėtingumo uždavinių imtį.

2. **Algoritmų realizacija:** Realizuoti tikslųjį pilno perrinkimo (angl. *Brute Force*) algoritmą (kaip etaloną) ir euristinį lygiagretųjį imituoto atkaitinimo (angl. *p-SA*) algoritmą.

3. **Technologinis palyginimas:** Įvertinti programavimo kalbos pasirinkimo įtaką skaičiavimų našumui, lyginant *NodeJS* ir *Rust* realizacijas.

4. **Kokybės ir našumo analizė:** Ištirti euristinio algoritmo tikslumą (RPD) ir vykdymo laiko priklausomybę nuo uždavinio dydžio.

5. **Daugiakriterinė analizė:** Nustatyti ryšius ir kompromisus tarp skirtingų optimizavimo tikslų: tuščios ridos, bendro atstumo ir transportavimo kainos.

Darbe naudojamas modulinis programinės įrangos sprendimas leidžia lanksčiai konfigūruoti optimizavimo tikslus bei generuoti įvairaus sudėtingumo problemas eksperimentams. Visas šio darbo metu sukurtas programinis kodas, įskaitant duomenų generavimo ir algoritmų realizacijas, yra viešai prieinamas [Šuk26].

1. Problemos aprašymas

Nagrinėjamas specifinis VRPPD uždavinys, kurio esmė – priskirti M užsakymų aibę N vilkikų parkui ir sudaryti optimalius maršrutus, tenkinančius griežtus logistikos apribojimus. Kiekvienas užsakymas susideda iš paėmimo taško (angl. *Pickup*) ir pristatymo taško (angl. *Delivery*).

Formaliai problemą apibrėžia šie komponentai:

- **Užsakymai (Orders):** Kiekvienas užsakymas turi unikalų identifikatorių, geografines paėmimo ir pristatymo koordinates bei krovinio užimamą vietą transporto priemonėje (angl. *load factor*). Laikoma, kad visų transporto priemonių talpa yra vienoda, o *load factor* nurodo užimamą dalį (pvz., 2 reiškia pusę vilkiko, t.y. $\frac{1}{2}$ talpos).

- **Vilkikai (Vehicles):** Transporto priemonės yra išsidėsčiusios skirtingose pradinėse lokacijose. Kiekvienas vilkikas turi nustatytą kainą už kilometrą, kuri gali skirtis priklausomai nuo vežėjo.

- **Maršrutas:** Tai seka lokacijų, kurias vilkikas turi aplankyti. Maršrutas privalo prasidėti vilkiko pradinėje vietoje, o paėmimo veiksmas privalo įvykti anksčiau nei atitinkamas pristatymas. Bet kuriuo metu vilkike esančių krovinių suma negali viršyti maksimalios talpos (krovinio faktorių suma ≤ 1).

Programinė realizacija leidžia optimizuoti pagal vieną iš trijų pasirinktų kriterijų (angl. *Optimization Targets*):

1. **EMPTY (Tuščia rida):** Minimizuojamas atstumas, kurį vilkikai nuvažiuoja be krovinio (nuo pradinio taško iki pirmo paėmimo bei tarp pristatymo ir kito paėmimo).

2. **DISTANCE (Bendras atstumas):** Minimizuojamas bendras visų vilkikų nuvažiuotas atstumas (tuščia rida + rida su kroviniu). Tai klasikinis VRP tikslas [TV02].

3. **PRICE (Kaina):** Minimizuojama bendra maršrutų kaina, atsižvelgiant į individualius vilkikų įkainius už kilometrą.

Tikslusis algoritmas (*Brute Force*) vieno vykdymo metu randa optimalius sprendinius visiems trims kriterijams, tuo tarpu euristinis algoritmas (p-SA) konfigūruojamas optimizuoti vieną pasirinktą tikslą per vykdymą.

2. Duomenų generavimas

Siekiant užtikrinti eksperimentų patikimumą ir rezultatų pritaikomumą realioje aplinkoje, sukurtas dviejų pakopų duomenų generavimo procesas. Duomenų generavimo posistemė sukurta naudojant *TypeScript* programavimo kalbą ir remiasi realiais logistikos maršrutų duomenimis.

2.1. Pirminių duomenų paruošimas

Pirmame etape (‘generate:data‘) naudojamas „sėklinis“ (angl. *seed*) duomenų rinkinys – CSV failas, kuriame pateikiami realūs krovinių pervežimo maršrutai. Iš šio failo sugeneruojami du pagrindiniai masyvai:

- **Užsakymai:** Sugeneruojama 11 000 užsakymų. Kiekvienam užsakymui, remiantis geografinėmis koordinatėmis, apskaičiuojami *Geohash* kodai (tikslumas 6 simboliai) greitesnei paieškai. Krovinių dydžiai (*load factor*) paimami iš pradinio duomenų rinkinio modelių.
- **Vilkikai:** Sugeneruojama 10 000 transporto priemonių. Siekiant imituoti realistinį scenarijų, kur transporto priemonės dažniausiai yra netoli potencialių krovinių, vilkikų pradinės lokacijos nėra visiškai atsitiktinės. Kiekvienas vilkikas generuojamas atsitiktinėje vietoje 200 km spinduliu aplink atsitiktinai parinkto užsakymo paėmimo tašką. Atstumai skaičiuojami naudojant sferinės geometrijos formules (Žemės spindulys $R \approx 6371$ km).

Šie duomenys išsaugomi JSON formatu (”orders_<timestamp>.json” ir ”vehicles_<timestamp>.json”) ir tarnauja kaip pagrindas tolimesniam specifinių uždavinių formavimui.

2.2. Uždavinių generavimas

Antrame etape (‘generate:problems‘) iš paruoštų masyvų formuojami konkretūs optimizavimo uždaviniai. Eksperimentams generuojami įvairaus dydžio mažos apimties uždaviniai.

Generavimo logika:

- Vykdomas ciklas per problemos dydžio parametrus: vilkikų skaičius (N) ir užsakymų skaičius (M) kinta intervale $[1, 7]$.
- Kiekvienai porai (N, M) sugeneruojama 10 skirtingų problemų. Tai leidžia išvengti statistinių nuokrypių, kurie galėtų atsirasti dėl vienos specifinės (pvz., labai „patogios“ arba „nepatogios“) geografinės konfigūracijos.
- Problemos sudaromos atsitiktinės atrankos būdu imant elementus iš pagrindinių užsakymų ir vilkikų masyvų.

Rezultatas – hierarchinė failų struktūra ‘problems/<N>_<M>/‘, kurioje saugomi JSON failai su apibrėžtais uždaviniais. Šis testavimo rinkinys (iš viso $7 \times 7 \times 10 = 490$ uždavinių) naudojamas tiek pilno perrinkimo algoritmui (kuris pajėgus išspręsti iki 7×7 dydžio uždavinius per priimtiną laiką), tiek euristinių algoritmų tikslumo validacijai.

3. Pilno perrinkimo algoritmas

Pilno perrinkimo (angl. *Brute Force*) algoritmas šiame darbe naudojamas kaip etaloninis metodas (angl. *baseline*), skirtas gauti garantuotai optimalius rezultatus mažos apimties uždaviniams. Pagrindinis šio metodo tikslas šiame kontekste – užtikrinti ir validuoti, kad žinomo modelio pritaikymas konkrečiam nagrinėjamam (euristiniam) algoritmui buvo atliktas tinkamai. Tai nustatoma lyginant sprendinius: laikoma, kad modelio adaptacija yra korektiška, jei modifikuoto algoritmo realizacija grąžina rezultatus, artimus pilno perrinkimo metodu gautam optimaliam sprendiniui.

Nors šio metodo laiko sudėtingumas yra eksponentinis, jis yra būtinas norint tiksliai įvertinti kuriamo algoritmo santykinę paklaidą ir patikrinti matematinio modelio apribojimų veikimą. Unikali šios realizacijos savybė – gebėjimas vieno vykdymo metu rasti globalius optimumus visoms trims tikslo funkcijoms (tuščiai ridai, bendram atstumui ir kainai), kas leidžia efektyviai sudaryti „Ground Truth“ duomenų rinkinį palyginamajai analizei.

3.1. Algoritmo logika ir matematinis pagrindas

Pilno perrinkimo algoritmas remiasi rekursinių būsenų medžio formavimu [CLR+22]. Problema sprendžiama dviem etapais, kurie vykdomi lygiagrečiai rekursijos metu:

1. **Užsakymų skaidymas (angl. *Partitioning*):** Visų galimų užsakymų aibių priskyrimas turimiems vilkikams.

2. **Maršruto sudarymas (TSP):** Kiekvienam vilkikui priskirtai užsakymų aibei randama optimali lankymo seka (atsižvelgiant į paėmimo ir pristatymo eiliškumą bei talpos apribojimus).

Algoritmo veikimo principas remiasi bitų kaukių (angl. *bitmasks*) technika [Sta21]. Kiekvienas užsakymas o_i atitinka i-ąjį bitą kaukėje. Rekursinė funkcija iteruoja per visus galimus poaibius (subkaukes) dar nepriskirtiems užsakymams. Tam naudojama efektyvi bitų operacija, leidžianti iteruoti tik validžius poaibius:

$$submask = (submask - 1) \& remainingMask \tag{1}$$

Šis metodas leidžia išvengti negalimų kombinacijų generavimo ir ženkliai pagreitina paiešką lyginant su naiviu iteravimu. Bendra algoritmo sudėtingumo klasė yra $O(V^N \cdot N!)$, kur V – vilkikų skaičius, o N – užsakymų skaičius.

3.2. Realizacija ir optimizacija (NodeJS ir Rust)

Siekiant įvertinti skirtingų technologijų įtaką skaičiavimų našumui ir užtikrinti etaloninių rezultatų tikslumą, algoritmas buvo realizuotas dviem būdais.

3.2.1. NodeJS realizacija

Pirminė versija sukurta *NodeJS* aplinkoje. Joje naudojamas „Branch and Bound“ principas [Cla99]: rekursijos gylis ribojamas, jei dalinis sprendinys jau viršija šiuo metu žinomą geriausią globalų rezultatą (angl. *pruning*). Sprendimo metu saugomi trys geriausi rasti variantai (kiekvienai tikslo funkcijai atskirai). Nors *JavaScript* V8 variklis yra greitas, rekursinės operacijos ir objektų kūrimas atmintyje riboja maksimalų sprendžiamą uždavinio dydį iki apytiksliai 6-7 užsakymų per priimtiną laiką ($< 5s$).

3.2.2. Rust realizacija ir validacija

Siekiant pagreitinti skaičiavimus ir išplėsti testuojamų duomenų imtį, buvo sugeneruota algoritmo versija *Rust* programavimo kalba. Ši realizacija buvo sukurta naudojant „Gemini 3 Pro“ didjįį kalbos modelį, automatiškai transpiluojant *NodeJS* logiką į *Rust*, išlaikant tą pačią algoritminę struktūrą, bet naudojant efektyvesnį atminties valdymą ir žemo lygio optimizacijas.

Atlikti lyginamieji testai parodė, kad *Rust* realizacija yra apytiksliai 3 kartus spartesnė už *NodeJS* versiją, o abiejų algoritmų gautos tikslo funkcijų reikšmės visiškai sutapo. Tai patvirtina, kad etaloninis metodas veikia korektiškai ir gali būti patikimai naudojamas vertinant, ar tiriamojo algoritmo rezultatai yra pakankamai artimi teoriniam optimumui.

4. Lygiagretusis imituoto atkaitinimo algoritmas

Kadangi VRPPD yra NP-sunkus uždavinys, didesnės apimties problemoms ($N > 7$) spręsti taikomas metaeuristinis metodas – lygiagretusis imituoto atkaitinimo algoritmas (angl. *Parallel Simulated Annealing*, p-SA). Šis algoritmas remiasi [WMZ+15] aprašytu metodu, adaptuojant jį specifiniam šio darbo modeliui ir panaudojant daugiagijį (angl. *multi-threaded*) vykdymą.

Skirtingai nei pilno perrinkimo algoritmas, p-SA vieno vykdymo metu optimizuoja tik vieną pasirinktą tikslą (EMPTY, DISTANCE arba PRICE). Tai leidžia algoritmui „sufokusuoti“ paiešką į konkrečios funkcijos minimizavimą.

4.1. Pradinio sprendinio generavimas (RCRS)

Imituoto atkaitinimo algoritmui kokybiškas pradinis sprendinys yra kritiškai svarbus. Šiame darbe naudojama modifikuota RCRS (angl. *Residual Capacity and Radial Surcharge*) euristika [Det01].

RCRS veikia „godžiuoju“ (angl. *greedy*) principu:

1. Užsakymai išmaišomi atsitiktine tvarka (siekiant diversifikacijos skirtingose gijose).
2. Kiekvienas užsakymas bandomas įterpti į visus įmanomus maršrutus, visose įmanomose pozicijose.
3. Skaičiuojama įterpimo kaina (angl. *insertion cost*). Realizacijoje ši kaina dinamiškai adaptuojama pagal pasirinktą optimizavimo tikslą:
   - **DISTANCE:** Kaina lygi maršruto ilgio pokyčiui Δd.
   - **PRICE:** Kaina lygi $\Delta d \times$ kaina_km.
   - **EMPTY:** Kaina lygi tuščios ridos pokyčiui, pridedant svertinį koeficientą atstumui nuo vilkiko pradinės lokacijos iki pirmo paėmimo taško (siekiant mažinti „tolimas“ tuščias keliones).
4. Pasirenkama pozicija su mažiausia kaina ir procesas kartojamas, kol visi užsakymai priskirti.

4.2. Lygiagretinimo strategija

Algoritmas realizuotas naudojant *NodeJS Worker Threads*. Pasirinkta lygiagretinimo strategija, primenanti „salų modelį“ (angl. *Island Model*) [AT02]:

- Sukuriamas darbininkų (angl. *workers*) masyvas, kurio dydis priklauso nuo procesoriaus branduolių skaičiaus (paprastai N_{cpu} arba daugiau).
- Kiekvienas darbininkas vykdo nepriklausomą imituoto atkaitinimo procesą su skirtingais pradiniais parametrais (pvz., šiek tiek varijuojančia pradine temperatūra).

- Periodiškai (kas nustatytą iteracijų skaičių, apibrėžtą ‘syncInterval‘ parametru) gijos keičiasi informacija. Geriausias rastas lokalus sprendinys siunčiamas kaimyninei gijai („pipeline“ topologija).
- Jei gija gauna geresnį sprendinį iš išorės, ji atnaujina savo dabartinę būseną ir šiek tiek „pakelia“ temperatūrą (angl. *re-heating*), kad išvengtų greito užstrigimo tame pačiame lokaliame optimume.

4.3. Kaimynystės operatoriai

Nauji kandidatai generuojami modifikuojant esamą sprendinį vienu iš trijų operatorių. Operatoriaus pasirinkimas yra tikimybinis, priklausantis nuo konfigūracijoje nustatytų svorių. Žemiau pateikiami kiekvieno operatoriaus algoritmai, atitinkantys programinę realizaciją.

4.3.1. Perkėlimo operatorius (Shift)

Šis operatorius (tikimybė $\approx 40\%$) paima atsitiktinį užsakymą iš vieno maršruto ir perkelia jį į kitą (arba tą patį) maršrutą. Kritinė šio algoritmo dalis – pristatymo taško (D) indekso generavimas intervale po paėmimo taško (P), taip užtikrinant pirmumo ribojimą.

1 algoritmas Operatorius: Shift

Require: $Solution$

1: $V_{src} \leftarrow$ Random$(Solution.NonEmptyVehicles)$
2: $O \leftarrow$ RandomOrder(V_{src})
3: RemoveOrder(V_{src}, O) {Pašalinami abu O taškai}
4: $V_{dest} \leftarrow$ Random$(Solution.AllVehicles)$
5: $idx_P \leftarrow$ Random$(0,$ Length$(V_{dest}) + 1)$
6: Insert$(V_{dest}, idx_P, O_{pickup})$
7: {Generuojamas indeksas po paėmimo taško:}
8: $idx_D \leftarrow$ Random$(idx_P + 1,$ Length$(V_{dest}) + 1)$
9: Insert$(V_{dest}, idx_D, O_{delivery})$

4.3.2. Sukeitimo operatorius (Swap)

Sukeitimo operatorius (tikimybė $\approx 30\%$) pasirenka du skirtingus vilkikus ir sukeičia jų užsakymus. Programinėje realizacijoje užsakymai ne įterpiami į atsitiktines vietas, o pridedami maršrutų pabaigoje. Tai garantuoja validumą be sudėtingų patikrų, o maršruto optimizavimą palieka kitoms iteracijoms.

2 algoritmas Operatorius: Swap

Require: $Solution$

1: $V_1, V_2 \leftarrow$ SelectDistinct$(Solution.NonEmptyVehicles)$
2: $O_1 \leftarrow$ RandomOrder(V_1)
3: $O_2 \leftarrow$ RandomOrder(V_2)
4: **if** $O_1 \neq O_2$ **then**
5: RemoveOrder(V_1, O_1)
6: RemoveOrder(V_2, O_2)
7: {Supaprastintas prijungimas prie maršruto galo:}
8: PushBack$(V_1, O_{2,pickup}, O_{2,delivery})$
9: PushBack$(V_2, O_{1,pickup}, O_{1,delivery})$
10: **end if**

4.3.3. Vidinis sumaišymas (Intra-Shuffle)

Šis operatorius (tikimybė $\approx 30\%$) skirtas optimizuoti vieną konkretų maršrutą, nekeičiant užsakymų pasiskirstymo tarp vilkikų. Jis iš naujo atsitiktine tvarka sugeneruoja paėmimo-pristatymo seką.

3 algoritmas Operatorius: Intra-Shuffle

Require: $Solution$

1: $V \leftarrow$ Random$(Solution.NonEmptyVehicles)$
2: **if** CountStops$(V) < 4$ **then**
3: **return** {Nėra prasmės maišyti 1 užsakymo}
4: **end if**
5: $Orders \leftarrow$ GetUniqueOrders(V)
6: ShuffleArray$(Orders)$
7: ClearStops(V)
8: **for all** $o \in Orders$ **do**
9: {Atkuriama validi seka P $\rightarrow$ D:}
10: PushBack(V, o_{pickup})
11: PushBack$(V, o_{delivery})$
12: **end for**

4.4. Parametrų derinimas (Tuning)

Kadangi euristinių algoritmų veikimas stipriai priklauso nuo parametrų (pradinės temperatūros, aušinimo koeficiento α, iteracijų skaičiaus), buvo sukurtas automatinis parametrų derinimo įrankis ('tune-psa').

Derinimas vykdomas naudojant 7×7 dydžio problemų aibę (didžiausią, kuriai žinomas tikslus globalus optimumas iš *Brute Force*). Kiekvienam optimizavimo tikslui (EMPTY, DISTANCE, PRICE) atskirai randama geriausia konfigūracija, minimizuojanti santykinę paklaidą (angl. *gap*) nuo globalaus

optimumo. Nustatyta, kad, pavyzdžiui, *PRICE* optimizavimui reikalinga aukštesnė pradinė temperatūra (1500) ir lėtesnis aušinimas (0.999) nei *EMPTY* tikslui, dėl didesnės kainos funkcijos variacijos.

5. Algoritmo pritaikymas ir pseudokodas

Nors p-SA algoritmas yra žinomas metodas, jo tiesioginis taikymas VRPPD uždaviniui su griežtais pirmumo (angl. *precedence*) ribojimais nėra trivialus. Šiame darbe realizuotas algoritmas buvo modifikuotas keliais esminiais aspektais, siekiant užtikrinti generuojamų maršrutų validumą ir išvengti gijų supanašėjimo.

5.1. Kaimynystės operatorių adaptacija

Standartiniai lokalios paieškos operatoriai (pvz., *2-opt* arba paprastas *Swap*) dažnai pažeidžia $t_{pickup} < t_{delivery}$ ribojimą, kai taškai sukeičiami vietomis. Programinėje realizacijoje pritaikytos šios strategijos:

- **Sukeitimas (angl. Swap).** Vietoje bandymo įterpti sukeistus užsakymus į optimalias pozicijas (kas reikalautų sudėtingo validavimo), realizuota supaprastinta logika: užsakymai pašalinami iš esamų maršrutų ir pridedami į kitų maršrutų **pabaigą**. Nors tai laikinai sumažina sprendinio kokybę (padidina atstumą), toks ėjimas garantuoja 100% validumą. Vėlesnėse iteracijose *Shift* ir *Shuffle* operatoriai optimizuoja šių „prikabintų“ užsakymų pozicijas.
- **Vidinis sumaišymas (angl. Intra-Shuffle).** Siekiant pagerinti maršruto kokybę po *Swap* operacijos, realizuotas operatorius, kuris atsitiktinai permaišo vieno maršruto užsakymų seką, kiekvieną kartą iš naujo suformuodamas validžią paėmimo-pristatymo grandinę.

5.2. Lygiagrečios sąveikos logika (Re-heating)

Šiame darbe realizuota *įtakos* (angl. Influence) logika su pašildymu:
Kai gija gauna geresnį sprendinį $S_{neighbor}$ iš kitos gijos:

1. **Sprendinys perimamas:** $S \leftarrow S_{neighbor}$.
2. **Pašildymas (Re-heating):** Temperatūra pakeliama iki nustatytos ribos ($T \leftarrow$ max$(T, 50)$). Tai suteikia algoritmui energijos „iššokti“ iš naujos pozicijos ir tyrinėti aplinką plačiau.

5.3. Algoritmo pseudokodas

Žemiau pateikiamas algoritmas (4 algoritmas) aprašo vienos darbinės gijos (angl. *Worker*) veikimo logiką, atitinkančią sukurtą programinę įrangą.

4 algoritmas Lygiagretaus imituoto atkaitinimo (p-SA) gijos veikimas

Require: $Problem, Target, Config$

$S \leftarrow$ RCRS$(Problem, Target)$ {Generuojamas pradinis sprendinys}
$T \leftarrow Config.initialTemp$
$S_{best} \leftarrow S$
$E \leftarrow$ CalculateEnergy$(S, Target)$
while $iteration < Config.maxIterations$ **and** $T > Config.minTemp$ **do**
  {— 1. Kaimyno generavimas —}
  $S_{new} \leftarrow$ GenerateNeighbor(S) {Shift / Lazy Swap / Shuffle}
  $E_{new} \leftarrow$ CalculateEnergy$(S_{new}, Target)$
  {— 2. Priėmimo kriterijus —}
  $\Delta E \leftarrow E_{new} - E$
  if $\Delta E < 0$ **or** Random$(0,1) <$ exp$(-\Delta E/T)$ **then**
    $S \leftarrow S_{new}$
    $E \leftarrow E_{new}$
    if $E <$ Energy(S_{best}) **then**
      $S_{best} \leftarrow S$
    end if
  end if
  {— 3. Sinchronizacija su kitomis gijomis —}
  if $iteration \pmod{SyncInterval} == 0$ **then**
    PostMessage(S_{best})
    if MessageReceived$(S_{neighbor}, E_{neighbor})$ **then**
      if $E_{neighbor} < E$ **then**
        $S \leftarrow$ Clone$(S_{neighbor})$
        $E \leftarrow$ CalculateEnergy$(S, Target)$
        $T \leftarrow$ max$(T, 50)$ {Pašildymas (Re-heating)}
      end if
    end if
  end if
  {— 4. Aušinimas —}
  $T \leftarrow T \times Config.coolingRate$
  $iteration \leftarrow iteration + 1$
end while
return S_{best}

6. Eksperimentiniai rezultatai

Šiame skyriuje pateikiama detali lyginamoji analizė tarp tiksliojo (angl. *Brute Force*) ir euristinio (p-SA) algoritmų. Tyrimo tikslas – įvertinti euristinio algoritmo gebėjimą rasti artimus optimaliems sprendinius per priimtiną laiką bei nustatyti jo patikimumo ribas.

6.1. Tyrimo metodika ir imtis

Eksperimentams naudojama sugeneruota duomenų aibė, kurią sudaro 490 unikalių VRPPD uždavinių. Uždavinių sudėtingumas varijuoja nuo 2 iki 14 objektų (bendra užsakymų ir vilkikų suma).

Siekiant užtikrinti statistinį rezultatų patikimumą, taikyta ši eksperimentų metodika:

- **Tikslusis algoritmas:** Kadangi *Brute Force* algoritmas yra deterministinis, kiekvienam iš 490 uždavinių jis buvo vykdomas po vieną kartą, vieno vykdymo metu randant optimalias reikšmes visiems trims tikslams (kainai, atstumui ir tuščiai ridai).
- **Euristinis algoritmas:** Kadangi p-SA yra stochastinis (atsitiktinumais paremtas) algoritmas, kiekvienas uždavinys kiekvienam iš trijų optimizavimo tikslų buvo sprendžiamas po **10 kartų**.
- **Rezultatų agregavimas:** Vykdymo laiko ir kokybės (RPD) vidurkiai skaičiuojami iš šių serijų rezultatų. Iš viso euristinio algoritmo testavimui buvo atlikta 490 problemų $\times$ 3 tikslai $\times$ 10 pakartojimų $= 14\,700$ nepriklausomų vykdymų.

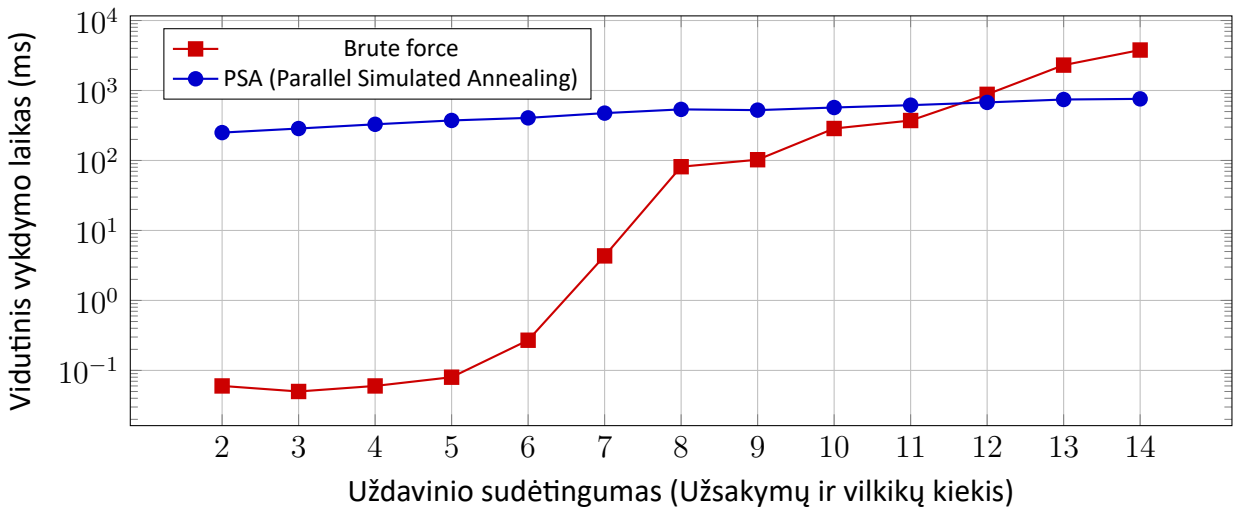
Visos laiko matavimo reikšmės pateikiamos milisekundėmis, o kokybės nuokrypiai lyginami su tiksliojo algoritmo gautu globaliu optimumu.

6.2. Algoritmų našumas ir laiko sąnaudos

Vienas svarbiausių VRPPD sprendimo aspektų yra algoritmo vykdymo laiko priklausomybė nuo uždavinio dydžio. 1 paveiksle vaizduojamas abiejų algoritmų vidutinis vykdymo laikas logaritminėje skalėje.

Kaip matyti grafike, *Brute Force* algoritmo (raudona kreivė) laiko sąnaudos auga eksponentiškai. Mažoms problemoms ($N < 7$) jis veikia beveik akimirksniu (< 5 ms), tačiau pasiekus 14 elementų dydį, vykdymo laikas šokteli iki beveik 4 sekundžių. Tai patvirtina teorinį $O(V^N)$ sudėtingumą ir parodo, kad realaus dydžio logistikos uždaviniams šis metodas yra netinkamas.

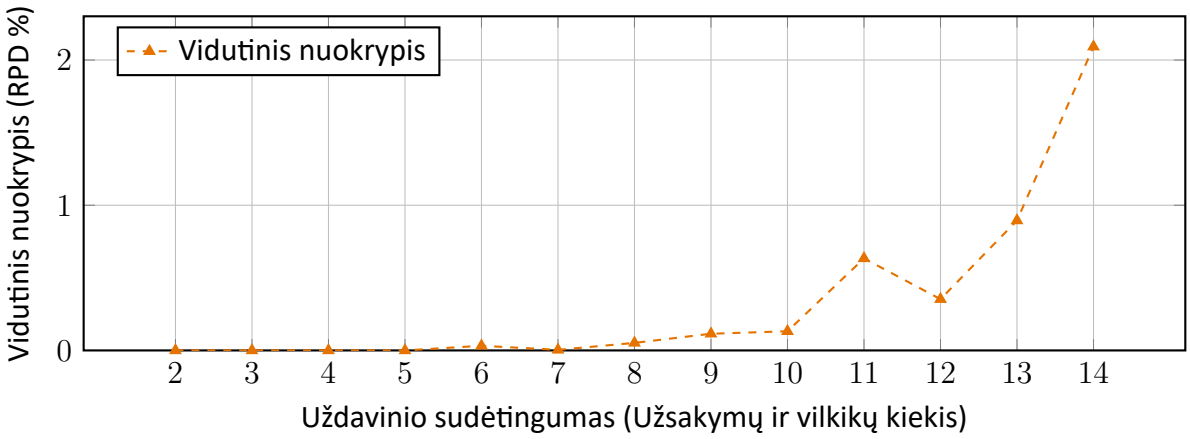
Tuo tarpu p-SA algoritmas (mėlyna kreivė) pasižymi aukštesnėmis pradinėmis laiko sąnaudomis (250 ms) dėl „overhead“, atsirandančio kuriant lygiagrečias gijas ir inicijuojant atminties struktūras. Tačiau didėjant uždavinio dydžiui, laikas auga tiesiškai arba labai lėtai. Ties dydžiu $N = 12$ įvyksta lūžio taškas, kuomet p-SA tampa greitesnis už pilną perrinkimą. Tai rodo p-SA algoritmo pranašumą sprendžiant didelės apimties uždavinius.



1 pav. *Algoritmų vykdymo laiko palyginimas (logaritminė skalė).*

6.3. Sprendinių kokybės analizė

Siekiant įvertinti euristinio algoritmo tikslumą, buvo skaičiuojamas vidutinis santykinis nuokrypis (angl. *Relative Percentage Deviation*, RPD) nuo globalaus optimumo, rasto pilno perrinkimo metodu. Rezultatai, gauti suvidurkinus 10 bandymų serijas kiekvienam taškui, pateikiami 2 paveiksle.



2 pav. *Sprendinių kokybės priklausomybė nuo uždavinio dydžio.*

Analizė rodo, kad:

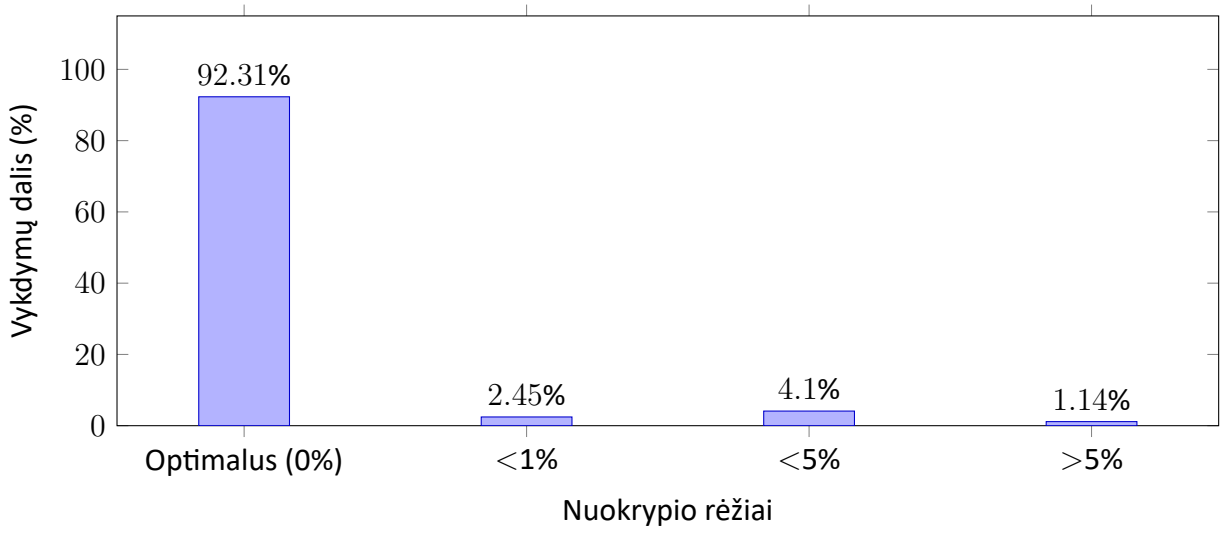
- Mažos apimties uždaviniams ($N \leq 5$) p-SA algoritmas veikia idealiai – RPD lygus 0%, t.y. visais bandymais randamas globalus optimumas.
- Vidutinio dydžio uždaviniams ($6 \leq N \leq 10$) pastebimas nežymus paklaidos atsiradimas, tačiau jis neviršija 0.15%, kas praktiniuose taikymuose yra nereikšminga.
- Didėjant kombinatoriniam sudėtingumui ($N > 12$), paklaida išauga iki 2%.

Iteracijų skaičiaus įtaka. Analizuojant algoritmo konvergavimo savybes (žr. 4 pav.), matyti, kad sprendinio kokybė stabilizuojasi gerokai anksčiau nei pasiekiamas iteracijų limitas. Tai lemia imituoto

atkaitinimo algoritmo prigimtis: temperatūrai nukritus, tikimybė priimti blogesnį sprendinį tampa artima nuliui, todėl algoritmas „užšąla“ lokaliame minimume. Paprastas iteracijų skaičiaus didinimas be pakartotinio „pašildymo“ (angl. *re-heating*) ar parametrų keitimo rezultatų reikšmingai nepagerintų, tik padidintų skaičiavimo laiką.

6.4. Algoritmo patikimumas

Ne mažiau svarbus rodiklis yra algoritmo stabilumas – tikimybė, kad paleidus algoritmą bus gautas geras rezultatas. 3 histograma apibendrina visų atliktų 14 700 euristinio algoritmo vykdymų rezultatus.



3 pav. *Euristikos sprendinių kokybės pasiskirstymas per visus vykdymus.*

Rezultatai rodo itin aukštą algoritmo patikimumą: net 92.3% visų vykdymų (įskaitant ir didžiausius uždavinius bei visas 10 pakartojimų serijas) rado globalų optimumą (0% nuokrypis). Tik maža dalis (1.1%) sprendinių nukrypo daugiau nei 5%. Tai leidžia teigti, kad pasiūlyta p-SA konfigūracija yra stabili ir nereikalauja didelio pakartojimų skaičiaus geram rezultatui gauti.

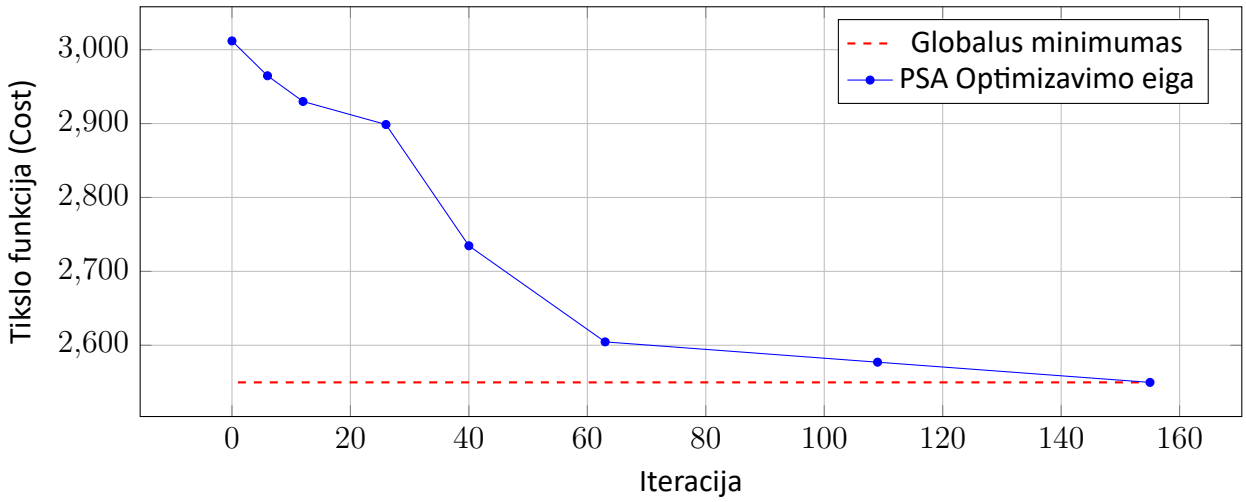
6.5. Konvergavimo charakteristikos

4 paveikslas demonstruoja tipinę p-SA algoritmo tikslo funkcijos (kainos/atstumo) mažėjimo eigą vieno vykdymo metu.

Grafike aiškiai matomi imituoto atkaitinimo etapai:

1. **Greitas nusileidimas (0-40 iteracijos):** Pradinėje stadijoje, esant aukštesnei temperatūrai, algoritmas priima įvairius sprendinius, tačiau greitai randa perspektyvias zonas, todėl tikslo funkcija sparčiai krenta.

2. **Stabilizacija (60-155 iteracijos):** Temperatūrai krentant, algoritmas pereina į lokalią paiešką. Kreivė tampa tolygesnė ir asimptotiškai artėja prie globalaus minimumo (raudona punktyrinė linija), kurį sėkmingai pasiekia ties 155 iteracija.



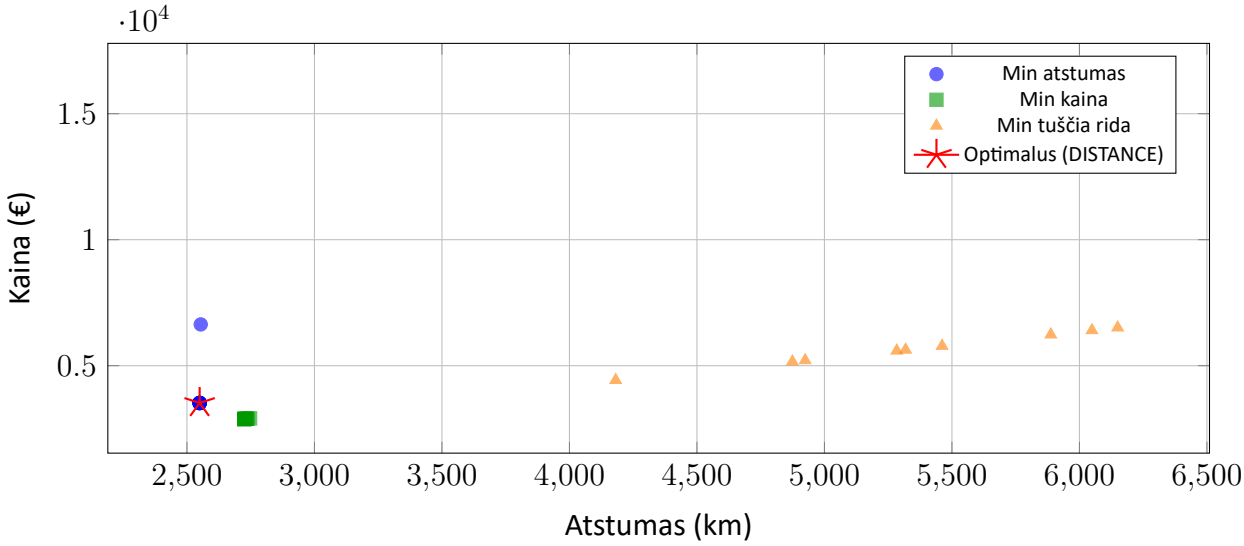
4 pav. *Algoritmo konvergavimo greitis (Tikslo funkcija vs Iteracijos).*

6.6. Daugiakriterinė analizė

VRPPD uždaviniuose dažnai susiduriama su prieštaraujančiais tikslais (angl. *conflicting objectives*). Siekiant ištirti p-SA algoritmo elgseną daugiakriterinėje erdvėje, buvo atlikta atskira eksperimentų serija su vienu, didžiausio kompleksiškumo uždaviniu.

Šiam uždaviniui algoritmas buvo paleistas po 10 kartų kiekvienam optimizavimo tikslui (atstumui, kainai ir tuščiai ridai). Žemiau pateikiamose diagramose vaizduojama **30 skirtingų euristinių sprendinių** (taškų) pasiskirstymas erdvėje, lyginant su *Brute Force* metodu rastu optimumu.

Taškai nuspalvinti pagal tai, koks buvo optimizavimo tikslas konkretaus vykdymo metu: **Mėlyna** – min. atstumas, **Žalia** – min. kaina, **Oranžinė** – min. tuščia rida. Raudona žvaigždutė žymi *Brute Force* metodu rastą geriausią sprendinį pagal atitinkamą X ašies parametrą.

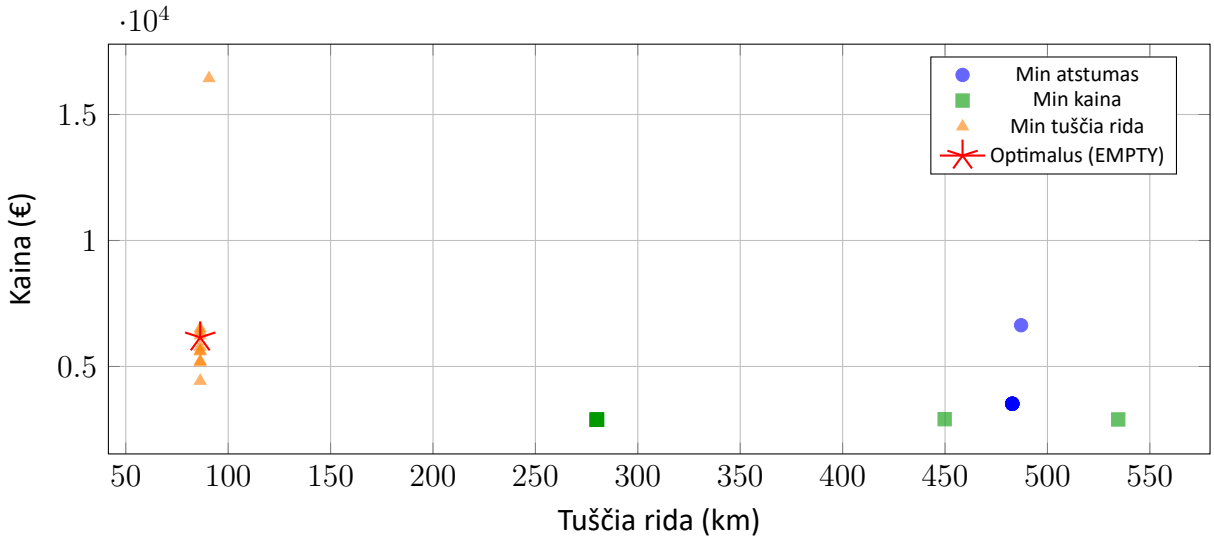


5 pav. *Atstumo ir kainos priklausomybė (Pareto analizė).*

Pirmasis grafikas (5 pav.) analizuoja ryšį tarp **atstumo ir kainos**.

- Mėlyni taškai (atstumo optimizavimas) telkiasi ties 2550 km riba, sutapdami su globaliu atstumo minimumu (raudona žvaigždute), tačiau jų kaina kartais yra aukštesnė (≈ 3500€).

- Žali taškai (kainos optimizavimas) rodo Pareto kompromisą: paaukojus šiek tiek atstumo (padidėjimas nuo $\approx 2550km$ iki $\approx 2730km$), pavyksta sumažinti kainą iki ≈ 2900€. Tai rodo, kad algoritmas geba rasti pigesnius maršrutus, tikėtina, pasirenkant pigesnius vilkikus, net jei jų maršrutas nėra pats trumpiausias.



6 pav. *Tuščio atstumo ir kainos priklausomybė (Pareto analizė).*

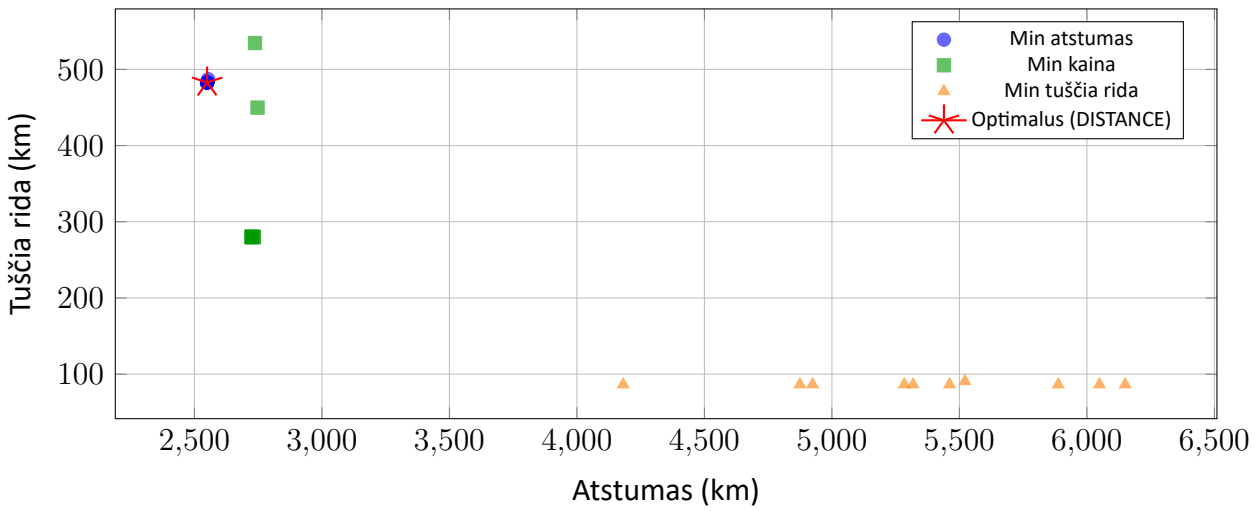
Antrasis grafikas (6 pav.) atskleidžia **tuščios ridos ir kainos santykį**.

- Oranžiniai taškai (min. tuščia rida) sudaro vertikalią liniją ties minimalia ≈ 86 km riba. Tai rodo, kad algoritmas visais 10 bandymų stabiliai rado maršrutus su minimalia tuščia rida, tačiau šių maršrutų kaina yra itin nepastovi ir dažniausiai aukšta (nuo ≈ 4400 € iki ekstremalių $16\,000$ €). Taip yra todėl, kad fokusuojantis tik į tuščią ridą, ignoruojamas bendras maršruto ilgis ir vilkikų įkainiai.

- Žali taškai (min. kaina) yra grafiko apačioje, demonstruodami mažiausius kaštus (≈ 2900 €). Svarbu pastebėti, kad optimizuojant kainą, algoritmas dažniausiai randa sprendinius su mažesne tuščia rida (≈ 280 km), nei optimizuojant bendrą atstumą (mėlyni taškai, ≈ 480 km).

Trečiasis grafikas (7 pav.) parodo koreliaciją tarp **bendro atstumo ir tuščios ridos**. Čia matoma, kodėl tuščios ridos optimizavimas (oranžiniai taškai) yra labai brangus: norint sumažinti važiavimą be krovinio iki minimumo (86 km), bendras maršruto ilgis išauga drastiškai (nuo 2550 km iki 4000-6000 km). Tai reiškia, kad vilkikai daro didelius lankus, kad visada važiuotų pilni.

Apibendrinant, šie rezultatai patvirtina, kad p-SA algoritmas sėkmingai aproksimuoja Pareto aibę ir leidžia logistikos planuotojams rinktis tarp skirtingų strategijų.



7 pav. *Atstumo ir tuščio atstumo priklausomybė (Pareto analizė).*

7. Tolimesni darbai

Šiame kursinio darbo etape buvo sėkmingai realizuotas ir ištirtas lygiagretusis imituoto atkaitinimo algoritmas (p-SA) bei sukurtas duomenų generavimo ir testavimo karkasas. Siekiant toliau vystyti projektą ir artėti prie praktinio taikymo realiose logistikos sistemose, numatomos šios pagrindinės plėtros kryptys:

1. **Koevoliucinio algoritmo (CEA) realizacija.** Remiantis literatūros apžvalga (pvz., [WC13]), numatoma realizuoti antrąjį euristinį metodą – koevoliucinį algoritmą. Tai leis atlikti išsamesnę lyginamąją analizę: palyginti trajektorijomis grįstą metodą (p-SA) su populiacijomis grįstu metodu (CEA).

2. **Didelės apimties uždavinių analizė ir „apatinės ribos“ vertinimas.** Kadangi tikslieji metodai (*Brute Force*) negali būti taikomi dideliems uždaviniams (pvz., $N > 50$), euristinių algoritmų kokybės vertinimas tampa sudėtingesnis. Ateityje planuojama lyginti rezultatus ne su nežinomu globaliu optimumu, o su teorine „apatine riba“ (angl. *Lower Bound*). VRPPD uždaviniuose apatinė riba gali būti apibrėžiama kaip visų užsakymų tiesioginių atstumų suma ($\sum dist(P_i, D_i)$), ignoruojant tuščią ridą. Tai leis objektyviai vertinti algoritmų efektyvumą net ir tūkstantinių užsakymų imtyse.

3. **Parametrų suderinimas (angl. Tuning) didesniems uždaviniams.** Dabartinė p-SA algoritmo parametrų konfigūracija buvo optimizuota naudojant 7×7 dydžio uždavinius. Tikėtina, kad didėjant problemos mastui, optimalūs parametrai (pvz., pradinė temperatūra, aušinimo greitis, gijų sinchronizacijos dažnis) keisis. Planuojama atlikti papildomus derinimo eksperimentus su didesnėmis imtimis, siekiant užtikrinti algoritmo stabilumą ir konvergavimo greitį.

4. **p-SA algoritmo perrašymas į Rust programavimo kalbą.** Atlikti eksperimentai su pilno perrinkimo algoritmu parodė, kad *Rust* realizacija buvo apie 3 kartus spartesnė už *NodeJS* versiją. Siekiant maksimalaus našumo ir geresnio daugiagijo vykdymo valdymo, planuojama migruoti p-SA algoritmą į *Rust* aplinką. Tai leistų per tą patį laiko limitą atlikti žymiai daugiau iteracijų ir potencialiai rasti geresnius sprendinius.

8. Išvados ir rezultatai

Šiame kursiniame darbe buvo sprendžiamas VRPPD optimizavimo uždavinys. Atlikus programinės įrangos kūrimo ir eksperimentinio tyrimo darbus, suformuluoti šie rezultatai ir išvados.

Gauti rezultatai

1. **Duomenų generavimo sistema.** Sukurtas dviejų pakopų duomenų generavimo mechanizmas ir suformuota 490 unikalių testinių problemų imtis (nuo 2 iki 14 objektų), paremta realiais geografiniais duomenimis.

2. **Algoritmų realizacija ir adaptacija.**

   - Realizuotas tikslusis pilno perrinkimo (angl. *Brute Force*) algoritmas, skirtas etaloninių rezultatų („Ground Truth“) gavimui.
   - Specifiniam VRPPD uždaviniui su griežtais eiliškumo ribojimais buvo pritaikytas žinomas lygiagretusis imituoto atkaitinimo (p-SA) metodas, naudojant RCRS euristiką pradiniam sprendiniui generuoti.
   - Pritaikytas žinomas lygiagretaus imituoto atkaitinimo algoritmas specifiniam uždaviniui spręsti.

3. **Technologinis našumas.** Pilno perrinkimo algoritmo realizacija *Rust* programavimo kalba leido pasiekti apytiksliai 3 kartus didesnį skaičiavimų greitį, lyginant su *NodeJS* versija, kas leido nežymiai išplėsti testuojamų uždavinių imtį.

Išvados

Remiantis gautais tyrimo rezultatais, daromos šios išvados:

1. **Metodo adaptacijos sėkmė.** Pasirinkto p-SA metodo adaptacija specifiniam VRPPD uždaviniui buvo sėkminga. Algoritmas pasižymi aukštu tikslumu: vidutinė santykinė paklaida (RPD) vidutinio dydžio uždaviniams neviršijo 1%, o 92,3% atvejų buvo rasti optimalūs sprendiniai.

2. **Algoritmų pritaikomumas.**

   - Tikslusis algoritmas dėl eksponentinio laiko augimo yra tinkamas tik mažos apimties uždaviniams ($N \leq 14$), tačiau yra būtinas euristinių metodų validavimui.
   - Euristinis p-SA algoritmas demonstruoja polinominį laiko augimą ir yra tinkamas didesnės apimties praktiniams uždaviniams spręsti, nes ties $N = 12$ riba tampa greitesnis už tikslųjį metodą, išlaikydamas aukštą sprendinių kokybę.

3. **Daugiakriterinė analizė.** Nustatyta, kad optimizavimas pagal vieną kriterijų dažnai neigiamai veikia kitus.

- Minimizuojant *tuščią ridą*, bendra maršruto kaina gali išaugti drastiškai (ekstremaliais atvejais net kelis kartus), nes vilkikai priversti daryti didelius lankstus.
- Minimizuojant *atstumą*, gaunamas trumpiausias kelias, tačiau ne visada pigiausias.
- *Kainos* optimizavimas pasirodė kaip efektyviausias „kompromisinis“ sprendimas, dažnai randantis balansą tarp tuščios ridos ir bendro kilometražo.

Literatūra ir šaltiniai

[AT02] E. Alba, M. Tomassini. „Parallelism and evolutionary algorithms“. Iš: *IEEE Transactions on Evolutionary Computation* 6.5 (2002), puslapiai 443–462.

[Cla99] J. Clausen. *Branch and bound algorithms-principles and examples*. Technical Report. Department of Computer Science, University of Copenhagen, 1999.

[CLR+22] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to algorithms*. 4-as leidimas. MIT press, 2022.

[Det01] J. Dethloff. „Vehicle routing and reverse logistics: the vehicle routing problem with simultaneous delivery and pick-up“. Iš: *OR Spectrum* 23 (2001), puslapiai 79–96.

[GJ90] M. R. Garey, D. S. Johnson. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. ISBN: 0716710455.

[Sta21] Stack Overflow. *How to iterate over subsets of a bitwise mask?* `https://stackoverflow.com/questions/67990260/how-to-iterate-over-subsets-of-a-bitwise-mask`. Prieiga per internetą: 2025-01-12. 2021.

[Šuk26] A. Šukelovič. *KDP-Algoritmai: Euristinių algoritmų ir globaliųjų optimumų lyginamoji analizė. Programinis kodas*. GitHub saugykla. 2026. URL: `https://github.com/AlexShukel/KDP-Algoritmai` (žiūrėta 2026-01-13).

[TV02] P. Toth, D. Vigo. *The vehicle routing problem*. SIAM, 2002.

[WC13] H.-F. Wang, Y.-Y. Chen. „A coevolutionary algorithm for the flexible delivery and pick-up problem with time windows“. Iš: *International Journal of Production Economics* 141.1 (2013). Meta-heuristics for manufacturing scheduling and logistics problems, puslapiai 4–13. ISSN: 0925-5273. `https://doi.org/https://doi.org/10.1016/j.ijpe.2012.04.011`. URL: `https://www.sciencedirect.com/science/article/pii/S0925527312001697`.

[WMZ+15] C. Wang, D. Mu, F. Zhao, J. W. Sutherland. „A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup-delivery and time windows“. Iš: *Computers & Industrial Engineering* 83 (2015), puslapiai 111–122.